

Project Plan - M2-1: AI and Machine Learning

Group 6

10 October 2025

1 Group Information

Group 6 Members						
Name	Omar Elfiki	Alexander Payn	Ashour Kariakous	Long Nguyen	Lorenzo Franca	Alexandru Mihăilă
Student ID	i6344736	i6385871	i6385235	i6389825	i6391129	i6400384

2 Research Question

Can we predict the time-to-threshold, denoting number of training steps and wall-clock time required for an agent to reach a performance threshold using initial configuration and early learning-curve signals?

3 Project Objectives

This project aims to build a predictive model that estimates the training time for a Unity ML-Agent to reach 80% of its maximum reward. By analyzing data collected from numerous training runs, the model will use initial configurations and early learning signals to make its predictions. Our complete methodology, code, and findings will be delivered in a well-documented GitHub repository.

To achieve this, we've broken down our plan into several key stages:

- Data Collection:** The foundation of our project will be a comprehensive dataset collected from Unity ML-Agents. We aim to conduct approximately **150 distinct training runs** using diverse algorithms and hyperparameter settings. For each run, we will systematically log all relevant information, including hardware specifications, agent parameters, runtime performance metrics (like RAM usage), and the outcomes.
- Model Development and Training:** Using the collected data, we will train a machine learning model to predict the “time-to-threshold.” We plan to start with baseline models and iteratively improve them as we expand our dataset and gain more insights.
- Performance Evaluation:** The success of our model will be measured rigorously. We will assess its predictive accuracy using appropriate **regression metrics on a held-out portion of our dataset**. This ensures the model generalizes well to new, unseen training configurations. We will define specific performance targets after our initial data exploration is complete.
- Documentation:** All methodologies, data collection processes, model architectures, and findings will be documented in a structured and detailed final report.

These objectives have been structured to follow the **S.M.A.R.T.** (Specific, Measurable, Achievable, Relevant, and Time-bound) framework and are directly relevant to answering our core research question regarding training dynamics.

4 Software Planning

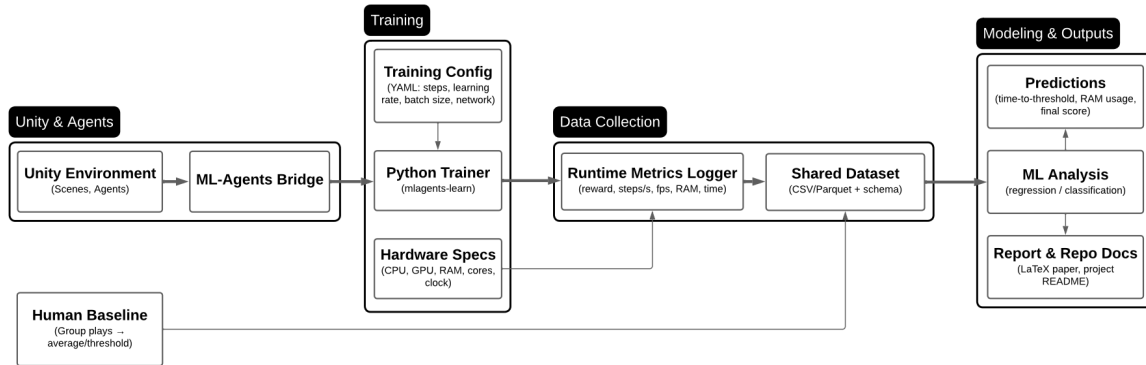


Figure 1: Block diagram

This section briefly explains how our team will use each software component to reach our project objective.

Unity

We will use Unity as the simulation engine where our agents train. Unity’s role is to run pre-made 3D simulations, provide the agent with observations and rewards, and execute the actions sent back from our Python code.

Python

To ensure reproducibility and cross-platform compatibility, we developed a setup script that automates the creation of a dedicated virtual environment. The script detects the user’s operating system, then installs the appropriate dependencies directly from a requirements file tailored to the detected platform. This approach guarantees that every team member works with the same compatible package versions and system configurations.

The result is a Python environment with all necessary libraries and packages pre-installed, allowing agent training to run immediately without manually managing packages. This reduces environment setup issues, improves reproducibility across machines.

ML-Agents

This is where the core machine learning takes place. The Unity ML-Agents toolkit provides a bridge between the Unity environment and a Python training process, configured via YAML training files. During training, the Unity environment sends observations and reward signals from the agent to Python. The Python trainer then processes this data, updates the learning model, and sends actions back to Unity, allowing the agent to gradually learn and improve its performance.

By default, ML-Agents uses the Proximal Policy Optimization (PPO) algorithm, a widely used reinforcement learning method, though other algorithms can also be configured.

TensorBoard (Training Visualization)

To analyze our agent’s learning, we will use TensorBoard. It integrates with ML-Agents to provide real-time visualizations of training data, such as rewards and policy loss. This allows us to monitor our experiments, debug effectively, and compare the performance of different training runs.

Git & GitHub

We’ll use Git to manage our project’s code. This enables us to work on different training scripts concurrently while preserving a comprehensive version history of our work.

Our project repository is hosted on GitHub and can be accessed here: [Project Repository](#)

The repository comprises multiple markdown files that assist in setting up the project, initiating training runs, and adhering to strict contribution and data logging guidelines to minimize conflicts and maintain the organization of training data.

The repository is also divided into sections based on the software components mentioned earlier. It includes a complete Unity project with example scenes from the ML-Agents package. Additionally, a training directory is present, which manages the Python side by providing a virtual environment initialization script that creates a new virtual environment with the necessary packages and dependencies. Lastly, documents and data directories are available, which store hardware information and results from training runs.

5 Data Collection

- Types of data to collect:
 - Hardware specifications: CPU, number of cores/threads, available RAM, OS details
 - Training configuration: DRL algorithm, learning rate, batch size, number of training steps
 - Runtime performance: training time, steps per second, RAM usage, CPU/GPU usage
 - Agent performance: success rate, training time
- How it will be collected:
 - Extracted logs generated by ML-Agents
 - Merging log data into a shared dataset
 - Names and data formats to allow combining results

6 Timeline

Week	Main Tasks	Responsible Person
1	Setup shared environment (Docker/Conda).	Ashour
	Create experiment configuration files.	Ashour
	Implement basic logging system.	Alexandru
	Design dataset structure and retention policy.	Alex
2	Add data validation and quality checks.	Alexandru
	Create diverse experiment list with algorithm variations.	Long
	Build automated experiment launch script.	Long
	Develop data cleaning and combination pipeline.	Alex
	Remove duplicates and calculate derived dataset values.	Alex
3	Test and verify logging system functionality.	Alexandru
	Execute short validation runs.	Long
	Perform initial dataset exploration and analysis.	Omar
	Define training and testing data split strategy.	Lorenzo
4	Build predictive models for runtime and memory usage.	Omar
	Setup basic experiment tracking system.	Lorenzo
	Create model training script with results output.	Lorenzo
5	Enhance logger with additional metrics and information.	Alexandru
	Expand experiment variety and environments.	Long

Week	Main Tasks	Responsible Person
6	Evaluate model accuracy and feature importance.	Omar
	Maintain dataset updates from new experimental runs.	Alex
	Stabilize logging system for extended training runs.	Alexandru
	Conduct longer training sessions for richer data.	Long
	Implement data versioning system.	Alex
	Experiment with model tuning and variations.	Omar
7	Add automated code and configuration checks.	Ashour
	Verify cross-machine experiment reproducibility.	Ashour
	Run feature importance and ablation studies.	Omar
	Improve experiment tracking and documentation.	Lorenzo
8	Ensure full results reproducibility across environments.	Ashour
	Maintain best model registry and performance tracking.	Lorenzo
	Final system validation and comprehensive documentation.	Ashour

Table 1: Project timeline and task distribution

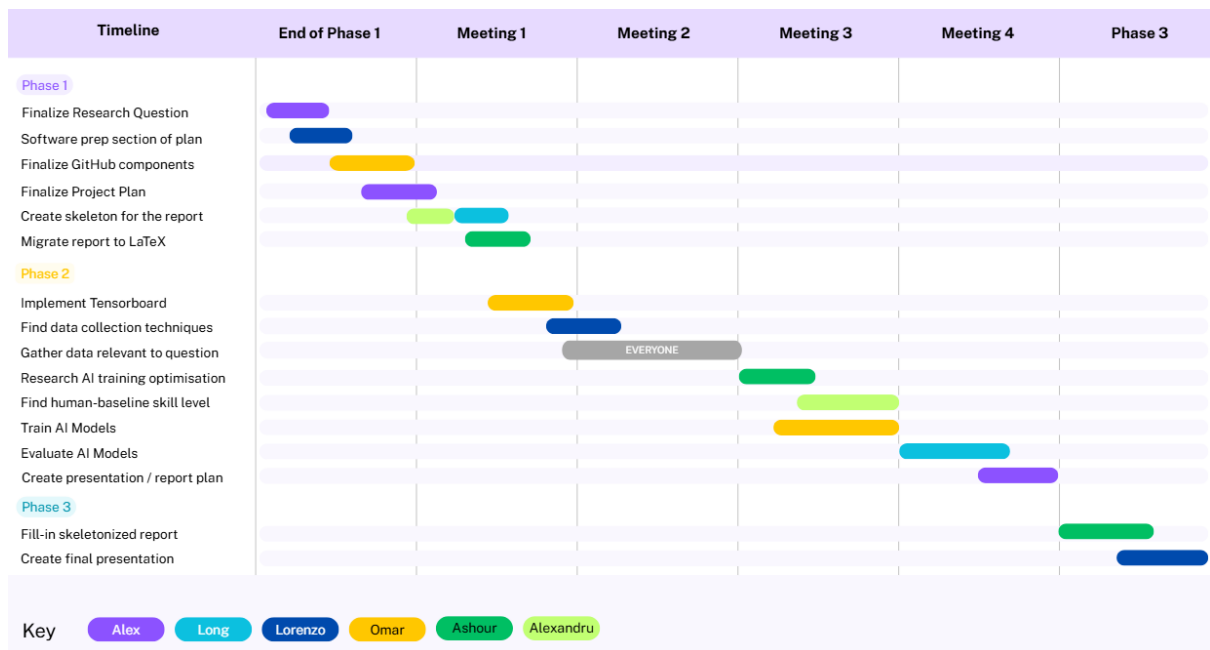


Figure 2: Gantt Chart

7 Risk Analysis

Risk	Likelihood	Impact	Mitigation
Environment setup issues (Python, Unity, Tensorboard, configuration / installation issues)	High	High	Create a shared installation guide, test setups early, support each other if errors occur
Git version conflicts / code loss	Medium	Very High	Use clear branching strategies, use commits with clear messages, keep backups on GitHub
ML scrips run unexpectedly (wrong optimization, poor results)	High	High	Test scrips earlier, independently; Validate regular outputs, adjust scope if results are too complex or too computationally expensive
Underestimation of time / deadlines	High	High	Distribute workload evenly among members, start earlier preparation
Poor communication / missed updates	Medium	Medium	More frequent check-ins, updates and reviews
Limited hardware resources	Medium	Medium	Reduce training steps, run smaller models, share data with other teams
Differences in Operating Systems / Setups	Very high	Medium	Test data on multiple available operating systems, create specialized installation guides

Table 2: Risk analysis with likelihood, and mitigation steps

References

- Unity Technologies. (2021). *ML-agents documentation*. <https://docs.unity3d.com/Packages/com.unity.ml-agents@3.0/manual/index.html>
- Unity Technologies. (2025). *Unity ml-agents toolkit [computer software]*. <https://github.com/Unity-Technologies/ml-agents>